

1. MVP Scope & Bounded Context

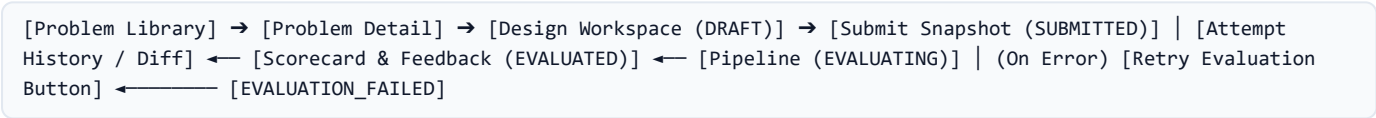
LLD Arena is an interactive low-level design practice platform. Its goal is to provide software engineers with an active architectural workbench that evaluates structural design quality, provides explainable diagnostic feedback, and tracks design improvement across iterations.

Curated Problem Library (4 Core Problems):

- Multi-Floor Automated Parking Lot:** Tiered vehicle spot allocation, gate barrier telemetry, hourly fee calculations, and capacity boards.
- Smart Multi-Car Elevator Dispatcher:** Hall call scheduling, state-driven car motion, door safety interlocks, and emergency overrides.
- State-Driven Smart Vending Machine:** Inventory slot matrices, multi-coin escrow validation, change return algorithms, and audit logging.
- On-Demand Ride Matching Service:** Spatial driver dispatching, surge multiplier pricing strategies, and trip state progression.

2. End-to-End User Flow & Attempt State Machine

The core user flow enforces an atomic, immutable submission lifecycle managed by an explicit finite state machine:



State Machine Transitions & Invariant Guarantees:

- DRAFT → SUBMITTED:** When the user submits, `AttemptEntity.submit()` validates invariants (non-empty model, valid type names). It freezes `draftDesign` into an immutable `SubmissionSnapshot`. Edits to drafts cannot mutate historical submissions.
- SUBMITTED → EVALUATING → EVALUATED:** Evaluation executes asynchronously. Results, radar dimensions, and rule findings are persisted in SQLite.
- EVALUATING → EVALUATION_FAILED → EVALUATING:** If an unhandled rule error or external timeout occurs, the snapshot is preserved, the state transitions to `EVALUATION_FAILED`, and the user can trigger `/api/attempts/:id/retry` without re-creating their design.
- Fork Revision (Attempt N → N+1):** Clicking "Revise Attempt" spawns a new `DRAFT` pre-populated with the previous submission, preserving parent-child lineage.

3. Domain Model & Key Classes

The domain layer strictly follows Domain-Driven Design (DDD) principles. Data bags and anemic models were eliminated in favor of rich domain entities encapsulating invariants:

Domain Entity / Class	Encapsulated Responsibilities	Key Invariants & Business Logic Enforced
<code>AttemptEntity</code> AGGREGATE ROOT	Lifecycle transitions, submission snapshotting, execution duration, and revision forking.	Enforces state machine transitions (DRAFT → SUBMITTED → EVALUATING → EVALUATED). Throws domain error if editing a submitted attempt.
<code>DesignEntity</code> DOMAIN ENTITY	Class, interface, and relationship AST representation; graph metric computations.	Calculates afferent/efferent coupling and inheritance depth. Validates unique identifiers and detects circular inheritance graphs.
<code>EvaluationPipeline</code> ORCHESTRATOR	Executes deterministic rules in isolated try/catches; invokes semantic evaluator; aggregates scores.	Adheres to the Open-Closed Principle (OCP). Dynamically aggregates score dimensions across registered rules without hardcoded categories.
<code>EvaluationRule</code> INTERFACE	Contract for pluggable deterministic structural checks (<code>evaluate(context)</code>):	Isolates rule failure: a crash in one rule records a diagnostic warning without terminating the entire pipeline.

Domain Entity / Class	Encapsulated Responsibilities	Key Invariants & Business Logic Enforced
	Promise<RuleResult>).	
SemanticEvaluator INTERFACE	Contract for qualitative feedback synthesis (evaluate(ctx, ruleResults)).	Implemented by LLMSemanticEvaluator (Gemini API with 6s timeout) and MockSemanticEvaluator (Dynamic offline heuristic engine).

4. Hybrid Evaluation Architecture (The Anti-Canonical Engine)

The evaluation pipeline rejects canonical string matching. Instead, it audits the **topological health of the candidate's design graph** using 5 deterministic rules and 1 semantic engine:

Rule / Evaluator	Category / Max	Deterministic Algorithm & Architectural Check
ClassSizeAndCohesionRule	Cohesion (Max: 20)	Detects God Classes (> 7 methods or combined size > 12). Flags single-class monolithic designs as critical failures (Score 0). Penalizes empty responsibility contracts.
CouplingMetricsRule	Coupling (Max: 20)	Constructs adjacency matrix. Detects bidirectional concrete coupling (A ↔ B). Flags high efferent coupling (Fan-out > 4). Penalizes 1-class systems lacking modular decomposition.
RelationshipIntegrityRule	Integrity (Max: 20)	Validates source/target type existence. Runs DFS three-color cycle detection on inheritance edges. Zeros score on circular hierarchies (A → B → A) and empty models.
RequirementsCoverageRule	Requirements (Max: 25)	Maps problem functional requirements against candidate's classes, methods, and responsibilities using tokenized intent matching. Penalizes anemic models lacking operational methods.
SolidPrinciplesRule	Extensibility (Max: 15)	Audits Dependency Inversion (DIP): flags concrete-to-concrete architectures lacking interface abstractions. Checks Interface Segregation (ISP) for fat interfaces (> 5 methods).
Dynamic Heuristic / LLM Evaluator	Semantic (35% Weight)	Analyzes domain entity completeness, pattern suitability, trade-off depth (scanning for senior architectural keywords), and future extensibility. Operates 100% offline or via Gemini REST API.

Scoring Synthesis Formula:

DeterministicScore

= (Σ Earned Rule Scores / Σ Max Rule Scores) × 100

OverallScore

= 0.65 × DeterministicScore + 0.35 × SemanticScore

Guards:

If the design contains 0 classes and 0 interfaces, OverallScore ≡ 0.

5. Validation Across 5 Design Archetypes

The evaluation engine was rigorously verified across 5 distinct architectural profiles:

- 1. **Good Canonical Design (Strategy Pattern):** Scored **96/100 (Exemplary)** with 0 critical findings.
- 2. **Poor Design (God Class Monolith):** Scored **41/100**. Correctly flagged with [CRITICAL] Monolithic God Class: "ParkingLotManager" absorbs the entire domain and Zero Architectural Decomposition.
- 3. **Structurally Different but Valid Design (State Pattern):** Scored **97/100 (Exemplary)**. Uses ParkingSlotState (AvailableSlotState, OccupiedSlotState). Proves the system does NOT penalize valid non-canonical solutions!
- 4. **Empty / Anemic Design:** Scored **0/100**. Awarded 0 points across all rules with 3 CRITICAL findings.
- 5. **Invalid Graph (Inheritance Cycle):** Scored **32/100**. Detected cycle A → B → A and completely zeroed out Structural Integrity (0/20).

6. Persistence Model (SQLite WAL)

The persistence layer uses SQLite via better-sqlite3 with Write-Ahead Logging (WAL) enabled. This architectural decision provides full ACID compliance, zero-latency synchronous reads, and file-backed persistence without requiring external Docker daemons or database connection strings.

```
CREATE TABLE problems (id TEXT PRIMARY KEY, slug TEXT UNIQUE, title TEXT, ... functional_requirements JSON); CREATE
TABLE attempts (id TEXT PRIMARY KEY, problem_id TEXT, attempt_number INTEGER, status TEXT, draft_design JSON,
submitted_snapshot JSON, failure_reason TEXT, ...); CREATE TABLE evaluations (id TEXT PRIMARY KEY, attempt_id TEXT
UNIQUE, overall_score REAL, score_dimensions JSON, rule_results JSON, semantic_feedback JSON, ...);
```

7. Pairwise Attempt Diff Engine

To prove learning progression, the system provides automated comparison between any two attempts (/api/attempts/compare?baseId=...&targetId=...):

- **Score Delta:** Computes net architectural score change (e.g., +15 points).
- **Dimension Progression:** Tracks dimensional shifts across Requirements, Cohesion, Coupling, Integrity, and Extensibility.
- **Resolved Violations:** Identifies rule failures present in Base Attempt that passed in Target Attempt.
- **Structural Net Diff:** Reports new classes introduced, interfaces extracted, and coupling cardinality changes.

8. Key Engineering Trade-offs & Non-Goals

Decision	Chosen Approach	Alternative Considered	Engineering Rationale
Architecture	Single-Port Monolith (Express 5 + React)	Microservices or Next.js SSR	Zero cold-starts, minimal operational surface, boots in 200ms, easy single-command verification (npm start).
Database	Embedded SQLite WAL	PostgreSQL / MongoDB container	Guarantees zero friction during evaluation—no Docker or network ports required. Repositories abstract persistence.
Diagram Engine	Declarative Mermaid.js UML	Interactive Drag-and-Drop Canvas	Focuses learner cognitive effort on object modeling and relationships rather than pixel-pushing coordinates.
Semantic Evaluator	Hybrid (Gemini REST + Dynamic Heuristic)	Pure LLM prompt evaluation	Pure LLMs are slow, non-deterministic, and fail offline. Dynamic heuristics guarantee 100% offline uptime and resilience.

Explicit Non-Goals (Scope Discipline):

To deliver a high-craft product within a 2-day scope, we intentionally excluded: (1) Multi-tenant user authentication/passwords, (2) Remote Docker code-execution sandboxes, and (3) Distributed Kafka/RabbitMQ brokers.

9. Verification & Quality Assurance

The platform contains **28 automated Vitest unit tests** running in < 600ms, validating attempt lifecycle states, graph cycle detection, god class penalization, failure recovery retries, and pairwise comparison metrics.